Documentation d'analyse énergétique

Erwan Didelot - Colin Dunand - Julia Mulder
Abel Prud'homme Lacroix - Baptiste Van Der Stracten

23 Janvier 2025

Contents

1 Introduction **2**

2 Méthodes de mesure **2**

3 Résultat des mesures **2**
3.1 Lors de la compilation 2
3.2 Lors de l'utilisation des tests automatiques 3
3.3 Estimation de notre impact énergétique tout au long du projet 4
3.4 Solutions pour réduire notre impact énergétique 4

4 Clean code - retour sur la conférence de Capgemini **4**

5 Conclusion **4**

1 Introduction

La responsabilité environnementale prend de plus en plus d'importance dans notre cursus car il faut former des ingénieurs conscients de l'impact de leurs décisions sur l'environnement. Dans ce rapport nous avons essayé d'évaluer la consommation énergétique de notre projet, en nous appuyant sur une analyse de la durée moyenne d'exécution des instructions. L'objectif est de chercher à réduire l'empreinte énergétique du projet sans réduire la qualité du rendu final.

2 Méthodes de mesure

Une première approche consiste à mesurer la consommation énergétique dès la phase de conception en se basant sur la durée des tests et des compilations. L'objectif est d'identifier les étapes les plus consommatrices d'énergie pour l'optimiser dès le début.

On peut aussi s'intéresser à la phase d'utilisation en évaluant l'impact des instructions inutiles générées par le compilateur. Ces instructions peuvent alourdir inutilement les performances énergétiques. Il faut donc les identifier et en réduire le nombre pour limiter leur influence sur la consommation globale.

3 Résultat des mesures

3.1 Lors de la compilation

Lors de la compilation du projet, le temps nécessaire pour compiler engendre une consommation énergétique annoncée par Maven, qui dépend notamment des caractéristiques des processeurs utilisés. Comme on doit compiler souvent le projet au cours d'une journée de travail, cela augmente forcément la consommation d'énergie.

La première compilation du projet affiche comme résultat le message suivant :

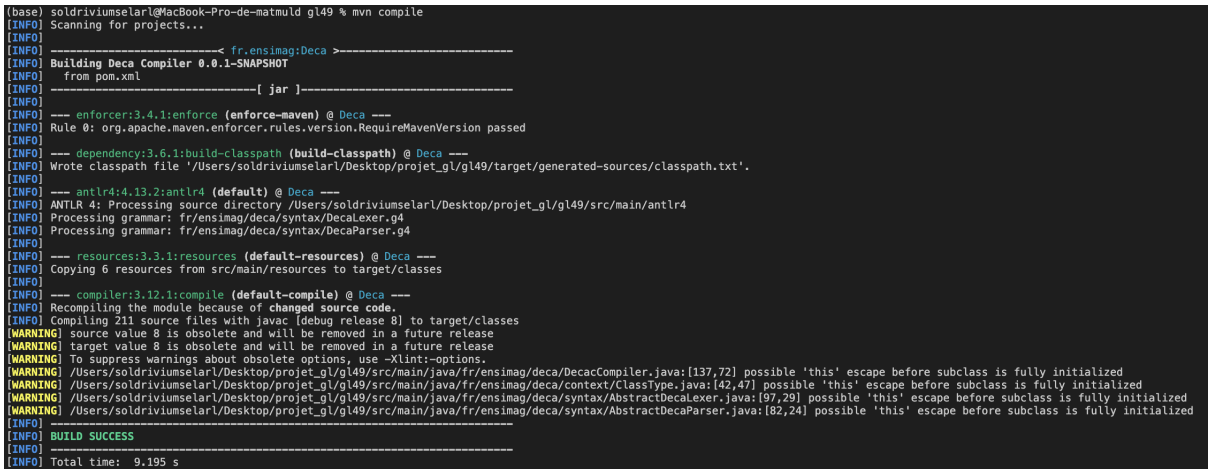


Figure 1: Première compilation du projet

Dans ce message, on peut voir plusieurs informations :

1. Temps total de compilation
Le temps total de compilation est de 9,195 secondes. Cela représente une donnée clé pour évaluer l'énergie consommée par le processus. Plus le temps de compilation est long, plus le processeur est sollicité, ce qui entraîne une consommation énergétique plus élevée.

2. Nombre de fichiers source compilés
Le processus compile 211 fichiers source Java. Le volume de fichiers compilés peut influencer la consommation énergétique : un grand nombre de fichiers entraîne une charge importante pour le compilateur.

3. Avertissements relevés

Les messages d'avertissement sont liés à des options obsolètes (comme source value 8 et target value 8), ils indiquent que le projet utilise des paramètres de compilation qui ne sont pas optimisés pour les versions récentes de Java. Cela peut avoir un impact négatif sur l'efficience énergétique. Passer à une version plus récente de Java permettrait d'améliorer la performance et réduire l'énergie nécessaire à la compilation et à l'exécution ultérieure des tests.

Lorsqu'on compile le projet ensuite pour la deuxième fois, on obtient le message suivant :

```
(base) soldriviumselarl@MacBook-Pro-de-matmuld gl49 % mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------------< fr.ensimag:Deca >-------------------------
[INFO] Building Deca Compiler 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- enforcer:3.4.1:enforce (enforce-maven) @ Deca ---
[INFO] Rule 0: org.apache.maven.enforcer.rules.version.RequireMavenVersion passed
[INFO]
[INFO] --- dependency:3.6.1:build-classpath (build-classpath) @ Deca ---
[INFO] Skipped writing classpath file '/Users/soldriviumselarl/Desktop/projet_gl/gl49/target/generated-sources/classpath.txt'.  No changes found.
[INFO]
[INFO] --- antlr4:4.13.2:antlr4 (default) @ Deca ---
[INFO] No grammars to process
[INFO] ANTLR 4: Processing source directory /Users/soldriviumselarl/Desktop/projet_gl/gl49/src/main/antlr4
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ Deca ---
[INFO] Copying 6 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.12.1:compile (default-compile) @ Deca ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.367 s
```

Figure 2: Deuxième compilation du projet

Dans ce message, on peut voir les informations suivantes :

1. Temps total de compilation

La compilation a pris 2,367 secondes, soit une réduction significative par rapport à la première exécution. Cela indique que des mécanismes de compilation incrémentielle et de cache ont fonctionné efficacement.

2.Étape ANTLR

La ligne "No grammars to process" montre que les grammaires n'ont pas été modifiées. ANTLR n'a donc pas réexécuté la génération de code pour les fichiers Lexer et Parser. Cette optimisation réduit la charge de calcul inutile.

3. Compilation Java

La ligne "Nothing to compile - all classes are up to date" indique qu'aucun fichier source Java n'a été modifié depuis la dernière compilation. Le compilateur Maven a bien détecté l'absence de changement des fichiers et a évité une recompilation complète, ce qui permet d'économiser des cycles CPU et de l'énergie.

4. Génération de fichiers inutiles évitée

La génération du fichier classpath ("Skipped writing classpath file") a été évitée, car aucune modification n'a été détectée. Cela montre que Maven est optimisé pour réduire les opérations inutiles.

3.2 Lors de l'utilisation des tests automatiques

La commande `mvn test` a permis d'exécuter automatiquement nos scripts de tests tout au long du projet pour s'assurer du fonctionnement de notre compilateur. L'exécution de cette commande prend du temps, 1 minute 56 secondes dans notre cas. Les tests sont une étape coûteuse en ressources, car ils impliquent des charges CPU pour exécuter le code de test et des accès mémoires pour charger les ressources nécessaires à leur exécution.

3.3 Estimation de notre impact énergétique tout au long du projet

Commençons par calculer l'énergie nécessaire à la compilation de notre projet.

On considère que nos ordinateurs possèdent des processeurs Intel i5 d'une puissance de base égale à 65 Watts d'après le site officiel de Intel. Notre projet compile en 2.4 secondes - on négligera la première compilation en considérant qu'on va faire beaucoup plus souvent les compilations d'après (on n'a pas besoin d'utiliser `mvn clean` à tous les coups).
Une compilation du projet coûte donc 2.4*65 = 156 Joules. On peut considèrer que l'on fait une cinquantaine de compilations par jour durant tout le projet, qui dure à peu près 28 jours, soit 218 kJ par personne.
Comme nous sommes une équipe composée de 5 personnes, on peut estimer la consommation énergétique de notre projet à 1000 kJ sur la durée complète du projet. Cela serait équivalent (d'après Wikipédia) à 16,7 minutes de chauffage par un radiateur de 1 000 W, ce qui est quand même conséquent, surtout que l'on n'a considéré pour l'instant que la compilation du projet.

Passons maintenant au calcul pour l'énergie nécessaire à l'exécution de nos tests. Nos tests sont exécutés en 116 secondes. Avec le même calcul que précédemment (en considérant que l'on n'exécute la commande `mvn test` qu'une trentaine de fois par jour), on obtient que l'exécution des tests pour notre projet consomme environ 32 000 kJ, soit 9 heures de chauffage par notre radiateur de 1 000 W.

Ces chiffres montrent bien l'impact énergétique que peut avoir l'informatique, car ils sont déjà significatifs sur un projet comme le nôtre qui ne dure qu'un mois, mais à l'échelle d'une entreprise, on se rend bien compte des conséquences le fait de ne pas optimiser son code peut avoir.

3.4 Solutions pour réduire notre impact énergétique

L'utilisation énergétique la plus conséquente est celle sur l'exécution des tests, c'est donc celle qu'il semble judicieux de limiter en priorité. Plutôt que d'utiliser `mvn test` très souvent, il serait mieux d'exécuter les tests les plus importants en priorité le plus souvent, et éviter de tout tester à chaque modification du projet. Il faudrait donc cibler les tests à retester en fonction de ce que l'on est en train de modifier.

4 Clean code - retour sur la conférence de Capgemini

Lors de la conférence organisée par Capgemini, le point central abordé était l'importance du "clean code" pour optimiser les performances des projets tout en réduisant leur impact énergétique. Une des recommandations majeures était d'éviter au maximum les redondances dans le code, c'est-à-dire que si du code est répété à plusieurs reprises, ça veut dire qu'il faut créer une fonction pour factoriser ce code. Cela permet de rendre le projet plus lisible et maintenable, mais aussi d'améliorer les performances en limitant les instructions inutiles générées par le compilateur.

Il nous semble aussi judicieux d'écrire le code au niveau le plus élevé possible dans les hiérarchies de classes. En plaçant les fonctionnalités communes au sommet des classes héritées, on limite les répétitions, mais il faut quand même que 'architecture reste claire.

Il pourrait aussi être intéressant de supprimer les instructions qui seraient inutiles au niveau assembleur, pour réduire le nombre d'instructions inutiles.

5 Conclusion

Faire l'analyse énergétique de notre projet nous a permis de plus prendre conscience de l'importance de réduire l'impact énergétique de nos activités en tant que futurs ingénieurs. Nos analyses ont révélé que la compilation et l'exécution des tests constituent des étapes très énergivores dans un processus de développement et cela même à l'échelle d'un petit projet comme le nôtre.